

Exercícios Complementares - Semana 01

Universidade Tecnológica Federal do Paraná (UTFPR), campus Apucarana
Curso de Engenharia de Computação
Disciplina de Programação Orientada a Objetos - POCO4A
Prof. Dr. Rafael Gomes Mantovani

-
1. Considere o seguinte código para a definição de um `Widget`:

```
def Widget:

    def Widget(self):
        self._msg = "Hello, I'm a widget!"

    def replace(self)
        index = self.index('w')
        self._msg[index] = 'g'

    def __str__(self):
        print 'My string is: ' + self._msg
```

Existem diferentes erros no programa acima (uma combinação de erros sintáticos e semânticos). Identifique e corrija pelo menos 4 dos erros.

2. Refaça a classe `Widget` acima em C++.

3. O código abaixo define uma classe `Radio` que implementa um modelo (muito) simples de rádio. Embora possa se criticar o realismo do modelo, há problemas mais sérios na modelagem. Especificamente, há 5 erros críticos com a sintaxe da classe definida. Identifique e os corrija.

```
class Radio( ):

    def init(self):
        self._powerOn = False
        self._volume = 5
        self._station = 90.7
        self._presets = [90.7, 92.3, 94.7, 98.1, 105.7, 107.7]
```

```

def togglePower(self):
    self._powerOn = not self._powerOn

def setPreset(self, ind):
    self._presets[ind] = _station

def gotoPreset(self, ind):
    self._station = self._presets[ind]

def increaseVolume(self):
    self._volume = self._volume + 1

def decreaseVolume(self):
    self._volume = self._volume - 1

def increaseStation(self):
    self._station = self._station + .2

def decreaseStation(self):
    self._station = self._station - .2

```

4. Refaça a classe `Radio` acima em C++.

5. Nosso design de `Televisão` (código da aula) é feito de maneira que a informação atual é mantida, mesmo que o aparelho seja desligado. Assim, quando ela é ligado novamente, as configurações de volume, canal, e mute continuam como estavam. Entretanto, muitas televisões reais tem um comportamento diferente: mesmo que estivessem mudadas quando foram desligadas, quando são ligadas novamente o volume não está mais mutado. Altere os métodos e funções da implementação de `Televisão` para adotar esse comportamento. **Obs:** Faça em ambas as linguagens (Python e C++).

6. Implemente um método `factoryReset()` para a classe `Televisão` que restaura todos os atributos (exceto `powerOn`) para os valores de fábrica. **Obs:** Faça em ambas as linguagens (Python e C++).

7. Defina uma classe `Conjunto`, que mantém o controle de uma coleção de objetos distintos. Internamente, use uma lista como uma variável de controle, mas sem permitir valores duplicados em seu conteúdo. A seguinte sugestão de design pode ser adotada:
O método para adicionar elementos deve inserir o elemento no conjunto se ele não existir.
O método para remover deve remover um elemento apenas se ele existir no conjunto. O

Conjunto
<code>__conteudo</code>
<code>Conjunto ()</code> <code>adicionar (valor)</code> <code>remover (valor)</code> <code>pesquisar (valor)</code>

método `pesquisar` retorna `True/False` se o elemento existe ou não dentro do conjunto. **Obs:** Faça em ambas as linguagens (Python e C++).

8. Incremente a classe `Televisão` para incorporar uma lista de canais favoritos como atributo. Três outros métodos precisam ser desenvolvidos:

- `addToFavorites( )` deve adicionar o canal atual na lista de canais favoritos, se não existir;
- `removeFromFavorites( )` deve remover o canal atual da lista de favoritos, se existir;
- `jumpToFavorite( )`, deve trocar o canal atual para um canal que existe na lista de favoritos. Especificamente, o canal favorito a ser trocado deve ser o próximo valor acima do valor corrente do canal atual. Se não existir um valor maior do que o canal corrente, usar o menor valor entre todos os favoritos. Se a lista de favoritos for vazia, o canal não deverá ser trocado.

Tome cuidado para que o método `jumpToFavorite( )` mantenha as propriedades do atributo `prevChannel` para ser usado nas chamadas de `jumpPrevChannel( )`. **Obs:** Faça em ambas as linguagens (Python e C++).